

PC40 – A26

Lab: From Sequential Neural-Network Training to Explicit Multithreading

Java – C++ – Free-threaded Python

J. GABER
gaber@utbm.fr

Important. This is not a Machine Learning lab. A complete sequential neural-network implementation is provided. Your job is to understand its computational structure, preserve its numerical behavior, and transform it into a correct and efficient multithreaded program.

Application: MNIST handwritten-digit recognition.

Main topic: explicit CPU multithreading, shared/private data, race conditions, mutual exclusion, barriers, reductions, granularity and performance.

Machine-learning prerequisite: none.

Contents

1	Learning outcomes	3
2	Files supplied	3
3	Preparation and smoke test	3
3.1	Dataset – required for all languages	3
3.2	Compile and execute	4
4	Part A – Neural networks in a nutshell	5
5	Part B – Establish the sequential baseline	5
6	Part C – Find the parallelism before writing threads	5
7	Part D – First explicit multithreaded implementation	6
8	Part E – Deliberately expose a race condition	6
9	Part F – Correctness with mutual exclusion	6
10	Part G – Better design: thread-local gradients	7
11	Part H – Synchronization and reduction	7
12	Part I – Correctness validation	7
13	Part J – Granularity and scalability	7
14	Part K – Optional advanced experiment: worker reuse	8
15	Optional HPC challenge – False sharing	8

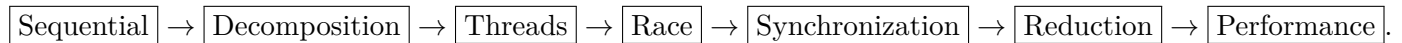
16 Language-specific guidance	8
16.1 C++	8
16.2 Java	8
16.3 Free-threaded Python	9
17 Debugging strategy	9
18 Required experimental table	9
19 Report structure	9
20 Organization and assessment	10
21 Minimum deliverables	10
22 Final synthesis questions	10

1 Learning outcomes

At the end of this lab, you should be able to

- identify independent work in a real application
- decompose a workload among workers
- create/start/join threads
- distinguish shared from private state
- explain and reproduce a race condition
- protect a critical section
- use synchronization correctly
- replace fine-grained locking by thread-local accumulation and reduction
- measure speedup and efficiency
- and explain why more threads do not automatically imply more performance.

in other words,



2 Files supplied

Choose **one language only** for your implementation.

```
PC40_Lab_Multithreaded_NN_v2/
|-- README.md
|-- data/download_mnist.py
|-- data/README.md
|-- cpp/nn_sequential.cpp
|-- java/NeuralNetworkSequential.java
|-- python/nn_sequential.py
'-- docs/PC40_Lab_Multithreaded_NN_v2.pdf
```

Do not modify the sequential file. Make a copy named `nn_multithreaded.cpp`, or `nn_multithreaded.py`, or `NeuralNetworkMultithreaded.java`.

3 Preparation and smoke test

3.1 Dataset – required for all languages

Regardless of the language chosen for the Lab (**C++**, **Java**, or **Python**), download the MNIST dataset once from the package root:

```
python3 data/download_mnist.py
```

This Python command is only a utility for downloading the dataset. It does **not** mean that Python must be chosen for the Lab.

The command creates the four standard MNIST IDX files in the `data/` directory. These binary files contain the handwritten digit images and their corresponding labels. The same files are then used by the C++, Java, and Python implementations.

If Internet access is unavailable, all three implementations support `-synthetic` for a quick functional smoke test.

Important. Use `-synthetic` only to check that your program compiles/runs correctly. Use the MNIST dataset for the performance measurements reported in your Lab report.

Visualizing the MNIST dataset

To better understand the data used by the neural network, you can visualize examples from the MNIST dataset.

If necessary, install `matplotlib`:

```
python3 -m pip install matplotlib
```

Then run:

```
python3 data/view_mnist.py
```

The script displays one example of each handwritten digit from 0 to 9.

Optionally, the visualization can be saved with:

```
python3 data/view_mnist.py --save mnist_examples.png
```

Each MNIST image contains $28 \times 28 = 784$ grayscale pixel values, which constitute the 784 inputs of the neural network.

Important. This visualization step is optional and independent of the programming language chosen for the Lab (C++, Java, or Python).

3.2 Compile and execute

C++17

```
cd cpp
g++ -O2 -Wall -Wextra -std=c++17 nn_sequential.cpp -o nn_sequential
./nn_sequential ../data --limit 12000
# smoke test: ./nn_sequential --synthetic --limit 2000
```

Java

```
cd java
javac NeuralNetworkSequential.java
java NeuralNetworkSequential ../data --limit 12000
# smoke test: java NeuralNetworkSequential --synthetic --limit 2000
```

Python

```
cd python
python3.14t nn_sequential.py ../data --limit 3000
# smoke test: python3.14t nn_sequential.py --synthetic --limit 500
```

For the multithreaded Python experiment, use a **free-threaded CPython build**. One convenient installation method is `uv python install 3.14t`, the python.org installers also provide a free-threaded option. A useful check is:

```
python3.14t -c "import sysconfig; print(sysconfig.get_config_var('Py_GIL_DISABLED'))"
```

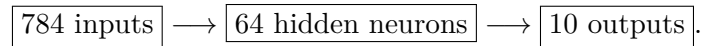
The expected value is 1. Record the exact Python version/build in your report. The Python reference uses `array.array('d')` for large numeric arrays to reduce shared Python-object reference-count contention. Measure T_1 and T_P with the **same interpreter build**; otherwise speedup is biased.

All three programs accept `-limit N`. The recommended default workload is larger for C++/Java than for pure Python because their per-sample execution costs differ. This is a runtime convenience, not an algorithmic difference. For every speedup curve, keep `-limit` fixed.

Checkpoint: Before continuing, the sequential program must compile/run and print epoch loss, training accuracy, test accuracy and training time.

4 Part A – Neural networks in a nutshell

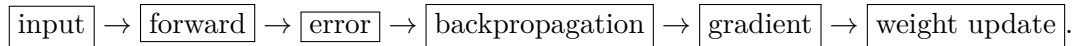
MNIST images contain $28 \times 28 = 784$ grayscale pixels. We use



The ten outputs represent digits 0–9. A neuron essentially computes

$$z_i = \sum_j w_{ij} x_j + b_i, \quad y_i = f(z_i).$$

The hidden layer uses ReLU, $f(z) = \max(0, z)$; the output uses softmax. Training follows



You are **not** asked to derive backpropagation. The important PC40 observation is that different samples of a mini-batch can compute gradient contributions independently while reading the same current weights.

Your task. Locate in the sequential source: weights/biases, forward computation, gradient computation, mini-batch loop, and weight update. Give the corresponding function/region names and one sentence describing each.

5 Part B – Establish the sequential baseline

Run the sequential implementation at least three times under similar machine conditions. Record median training time T_{seq} , accuracy, CPU model/core count, OS, compiler/JVM/Python version, and optimization flags.

Your task. Explain why a parallel program must be compared against a stable sequential baseline. Do not change dataset size, epoch count, batch size, optimization level or machine conditions between configurations.

Optional: instrument gradient computation and weight update separately. Do not include dataset loading in training-time comparisons.

6 Part C – Find the parallelism before writing threads

For a batch $\mathcal{B} = \{x_0, \dots, x_{B-1}\}$, each sample produces a gradient contribution g_i and

$$G = \sum_{i \in \mathcal{B}} g_i.$$

With four workers: $T_0 : \mathcal{B}_0, T_1 : \mathcal{B}_1, T_2 : \mathcal{B}_2, T_3 : \mathcal{B}_3$. Each worker performs forward + backpropagation for its assigned samples.

Your task. Classify weights, input samples, labels, local gradients, global gradient, loss/accuracy counters, and updated weights as **shared read-only during gradient computation, thread-private, or shared writable**. Justify each classification.

Your task. For batch size 64 and $P = 4$, give the sample interval handled by each worker. Then give a general partitioning method that also works when B is not divisible by P .

7 Part D – First explicit multithreaded implementation

Use the lecture pattern:

```
create workers → start all workers → wait for all workers → collect results.
```

Never start one worker and immediately wait for it inside the same loop.

C++	Java	Python
<code>std::thread</code>	<code>Thread</code>	<code>threading.Thread</code>
<code>join()</code>	<code>join()</code>	<code>join()</code>
<code>std::mutex</code>	<code>synchronized/Lock</code>	<code>threading.Lock</code>

Your task. Create a multithreaded copy and add configurable worker count P . First parallelize sample processing inside each mini-batch. Keep algorithm, initialization seed, dataset size, batch size, learning rate and epochs unchanged.

Checkpoint: Test $P = 1$ first. A one-worker version separates restructuring bugs from concurrency bugs.

8 Part E – Deliberately expose a race condition

Suppose workers directly accumulate into shared state:

```
global_gradient[i] += contribution;
shared_loss += sample_loss;
shared_correct += is_correct;
```

An expression such as `x += a` is conceptually read $\rightarrow$ modify $\rightarrow$ write. Two threads can interleave these steps and lose an update.

Your task. Implement or isolate a deliberately unsafe shared accumulation. Run it repeatedly. Compare final loss/accuracy and, where practical, a checksum of the gradient or weights. Explain why a race may exist even if one run appears correct.

Important. A race condition is a correctness defect. A timing-dependent bug is not disproved by a run that happens to produce the expected result. In C++, a data race causes **undefined behavior**; it is not merely a possible lost update.

C++ diagnostic: ThreadSanitizer

For the C++ path, use ThreadSanitizer on the deliberately unsafe version:

```
g++ -std=c++17 -O1 -g -fsanitize=thread -pthread \
    nn_multithreaded.cpp -o nn_tsan
./nn_tsan --synthetic --limit 500
```

A sanitizer report is stronger evidence than “it failed once”. After correcting the race, run the sanitizer again.

9 Part F – Correctness with mutual exclusion

Protect the shared update with a lock/mutex and keep the protected region as small as possible.

Your task. Implement a correct lock-based version. State precisely what invariant the lock protects. Measure its time. Compare with the unsafe version and sequential baseline. Explain contention and synchronization overhead.

Discuss: What if the lock covers the entire sample computation? What if it is acquired for every scalar gradient update? Why can both destroy performance?

10 Part G – Better design: thread-local gradients

A better design avoids frequent shared writes. Worker p owns a private accumulator G_p :

$$T_0 \rightarrow G_0, \quad T_1 \rightarrow G_1, \quad \dots, \quad T_{P-1} \rightarrow G_{P-1}.$$

During expensive forward/backward computation, all workers read the same weights but write only to private gradient arrays and private statistics.

Your task. Replace fine-grained shared gradient updates by one local gradient per worker. Also keep local loss and local correct-count values. Confirm that the main computation no longer requires a mutex.

Your task. Estimate the additional memory cost of P local gradient copies. The supplied network has $784 \times 64 + 64 + 64 \times 10 + 10 = 50\,890$ trainable parameters, about 199 KiB per float32 gradient copy. For $P = 16$, this is only about 3.1 MiB, so the overhead is negligible here. Explain why the conclusion could change for a model with 10^8 parameters and $P = 64$. Do not invent a trade-off that is not significant for this small network.

11 Part H – Synchronization and reduction

Weights must not be updated until every worker has completed the current mini-batch:

$$\{T_p \rightarrow G_p\}_{p=0}^{P-1} \rightarrow \boxed{\text{synchronization}} \rightarrow G = \sum_p G_p \rightarrow W \leftarrow W - \eta G/B.$$

A `join()` can be the synchronization point in a simple create-per-batch design. A reusable-worker design normally needs a barrier or equivalent coordination.

Your task. Implement reduction of all local gradients into one batch gradient, then perform exactly one weight update for that batch. Combine local loss/correct counters in the same reduction phase.

Your task. Explain what could happen if one worker starts the next batch with new weights while another worker is still computing the previous batch with old weights.

Checkpoint: The desired design is now race-free by construction: shared model weights are read-only while workers compute; workers write private accumulators; the update occurs only after all workers complete.

12 Part I – Correctness validation

Parallel floating-point reductions can occur in a different order, so bit-for-bit identity is not always expected. The program must nevertheless remain numerically consistent.

Your task. For $P = 1, 2, 4$, compare epoch losses, final test accuracy, and a simple checksum such as the sum of all weights. Define reasonable tolerances and explain small floating-point differences. Large or unstable differences indicate a bug.

13 Part J – Granularity and scalability

Test $P = 1, 2, 4, 8$ and, if meaningful on your machine, 16. Record physical/logical core counts. Keep all other parameters fixed. Optionally repeat for batch sizes 32, 64, 128 and 256. For each P compute

$$S_P = \frac{T_1}{T_P}, \quad E_P = \frac{S_P}{P}.$$

Here T_1 should be the one-worker version of the same parallel structure for the cleanest scaling study; report the original sequential baseline separately.

Your task. Produce: (1) execution time versus thread count; (2) speedup versus thread count, including ideal $S_P = P$; (3) a table with P, T_P, S_P, E_P . Use at least three runs per configuration and report the median.

Discuss thread-management overhead, synchronization, reduction cost, load balance, memory/cache effects, physical versus logical cores, and the serial fraction (Amdahl's law).

14 Part K – Optional advanced experiment: worker reuse

Creating/destroying P threads for every mini-batch can be expensive. Optionally create a fixed set of workers once and reuse them. You need a safe mechanism to signal new work and synchronize before each weight update.

Your task. Compare create-per-batch versus reusable workers. Do not attempt this until the simpler version is correct. Explain whether the extra complexity produces measurable benefit. Use established primitives rather than home-made flag barriers: C++20 `std::barrier` (or C++17 `condition_variable`), Java `ExecutorService/CyclicBarrier`, or Python `threading.Barrier/ThreadPoolExecutor`.

15 Optional HPC challenge – False sharing

Thread-private variables can still interfere at the cache-coherence level if they occupy the same cache line. For example, contiguous per-thread counters such as `correct[p]` or `loss[p]` may exhibit false sharing.

Your task. If your platform permits it, compare compact per-thread counters with padded/aligned storage. In C++, for example, try a record with `alignas(64)`. Explain why logically independent variables can still reduce one another's performance. Treat this as an optional experiment; do not sacrifice correctness work to complete it.

16 Language-specific guidance

16.1 C++

Include `<thread>` and `<mutex>`. Store worker objects until all have been started, then join them in a second loop. Prefer RAII locking such as `std::lock_guard<std::mutex>` for the lock-based experiment. Compile the parallel version with:

```
g++ -O2 -std=c++17 -pthread nn_multithreaded.cpp -o nn_multithreaded
```

Do not accidentally copy the whole network into each thread; workers should share the current model for read access while computing a batch.

16.2 Java

The JVM uses JIT compilation. Perform at least one warm-up run before recording timings, or explicitly exclude the first run, and use the same timing protocol for every P .

Create all `Thread` objects, start all of them, then join all of them. For the lock experiment, use either `synchronized` or `java.util.concurrent.locks.Lock`. Avoid making the whole training function `synchronized`: that would serialize the useful work.

16.3 Free-threaded Python

Use `threading.Thread` and `threading.Lock`. This lab expects a free-threaded CPython build for CPU-bound scaling. Do not replace threads by `multiprocessing`: that would answer a different PC40 question. Record how you verified your interpreter configuration. The same start-all / join-all discipline applies.

17 Debugging strategy

When the parallel version fails, do not change many things at once. Use this order:

1. Confirm the untouched sequential reference still works.
2. Run the parallel structure with $P = 1$.
3. Use a tiny dataset or `-synthetic` for fast debugging.
4. Check sample ranges: no missing or duplicated samples.
5. Check that each worker has distinct local gradient storage.
6. Check that all workers finish before reduction/update.
7. Check that weights are not modified during worker gradient computation.
8. Only after correctness is established, measure performance.

A fast wrong program is not a successful parallel program.

18 Required experimental table

Use a table similar to the following and add rows if needed.

Threads P	Median T_P (s)	Speedup S_P	Efficiency E_P	Test accuracy (%)
1		1.00	1.00	
2				
4				
8				
16				

19 Report structure

For **each exercise/part**, structure the report as follows:

1. **Objective:** state what the exercise is intended to demonstrate.
2. **Answers:** answer the conceptual questions and explain design decisions.
3. **Results:** provide relevant code excerpts, terminal screenshots and requested plots/tables, with comments and explanations. Do not paste pages of source code.
4. **Conclusion:** recall the objective and state the main technical message learned from the experiment.

The report must be written in English.

20 Validation of your work

At the end of the second lab session devoted to this topic, the instructor will check the execution of your code and ask you a few questions to verify the correctness of your implementation, your understanding of the main concepts used in this Lab, and the parallelization choices you made.

21 Final synthesis questions

Answer concisely but precisely:

- Q1. Why was the mini-batch a natural unit for exposing parallelism?
- Q2. Why are shared data not automatically dangerous? Give a read-only shared example from this lab.
- Q3. Why is `x += a` on shared state a potential race?
- Q4. Why can a mutex make a program correct but slow?
- Q5. Why do thread-local gradients reduce contention?
- Q6. Why is synchronization required before updating weights?
- Q7. What is a reduction in this application?
- Q8. Why does speedup normally deviate from $S_P = P$?
- Q9. Why can using more threads than useful hardware contexts hurt performance?
- Q10. Which parts of your manual solution do you expect OpenMP to simplify?

The message to remember

Parallel programming is not “add threads”.

A good parallel program requires decomposition, correct management of shared state, synchronization only where needed, reduction of partial results, appropriate granularity, and experimental performance analysis. The neural network is the workload; these parallel-programming principles are the subject of the lab.